dev.to

How to deploy Llama-3.1-Nemotron-70B-Instruct on a Virtual Machine in the Cloud?

Ayush kumar

8–10 minutes

70B Instruct in the Cloud

NVIDIA has developed a large language model called Llama-3.1-Nemotron-70B-Instruct, which aims to increase the usefulness of responses produced by large language models (LLM). In a number of auto-alignment benchmark tests, including GPT-4-Turbo MT-Bench, AlpacaEval 2 LC, and Arena Hard, this model performed exceptionally well. It is trained on the Llama-3.1-70B-Instruct model using HelpSteer2-Preference prompts, Llama-3.1-Nemotron-70B-Reward, and RLHF (more precisely, the REINFORCE algorithm).

In addition to demonstrating how NVIDIA has improved generated support for general domain instructions through technology advancements, this model provides a format for model conversion that is compatible with the Hugging Face Transformers library and allows for free hosted inference via the NVIDIA build platform.

Leaderboard

The following leaderboard has no style control.

(Updated: 10/14)

NVIDIA.

o1-mini-2024-09-12	score: 92.0	95% CI: (-1.2, 1.0)
o1-preview-2024-09-12	score: 90.4	95% CI: (-1.1, 1.3)
llama-3.1-nemotron-70b-instruct	score: 84.9	95% CI: (-1.7, 1.8)
gpt-4-turbo-2024-04-09	score: 82.6	95% CI: (-1.8, 1.5)
yi-lightning	score: 81.5	95% CI: (-1.6, 1.6)

```
claude-3-5-sonnet-20240620     | score: 79.3  | 95% CI: (-2.1, 2.0)
gpt-4o-2024-05-13              | score: 79.2  | 95% CI: (-1.9, 1.7)
gpt-4-0125-preview             | score: 78.0  | 95% CI: (-2.1, 2.4)
qwen2.5-72b-instruct           | score: 78.0  | 95% CI: (-1.8, 1.8)
gpt-4o-2024-08-06              | score: 77.9  | 95% CI: (-2.0, 2.1)
athene-70b                     | score: 77.6  | 95% CI: (-2.7, 2.2)
gpt-4o-mini                    | score: 74.9  | 95% CI: (-2.5, 1.9)
gemini-1.5-pro-api-preview     | score: 72.0  | 95% CI: (-2.1, 2.5)
mistral-large-2407             | score: 70.4  | 95% CI: (-1.6, 2.1)
llama-3.1-405b-instruct-fp8    | score: 69.3  | 95% CI: (-2.4, 2.2)
glm-4-0520                     | score: 63.8  | 95% CI: (-2.9, 2.8)
```

Transformer Design: The model is skilled at comprehending context and producing pertinent responses because of the transformer architecture, which allows it to grasp long-range dependencies in text.

Multi-Head Attention: This feature makes it possible for the model to concentrate on various input elements at once, improving its comprehension of intricate queries and enabling it to generate nuanced results.

Layer Normalization: Throughout the model, layer normalization is used to enhance convergence rates and stabilize training, leading to quicker and more effective learning.

Evaluation Metrics

As of 1 Oct 2024, Llama-3.1-Nemotron-70B-Instruct performs best on Arena Hard, AlpacaEval 2 LC (verified tab) and MT Bench (GPT-4-Turbo)

Model	Arena Hard	AlpacaEval	MT-Bench	Mean Response Length

Details	(95% CI)	2 LC (SE)	(GPT-4-Turbo)	(# of Characters for MT-Bench)
Llama-3.1-Nemotron-70B-Instruct	**85.0** (-1.5, 1.5)	**57.6** (1.65)	**8.98**	2199.8
Llama-3.1-70B-Instruct	55.7 (-2.9, 2.7)	38.1 (0.90)	8.22	1728.6
Llama-3.1-405B-Instruct	69.3 (-2.4, 2.2)	39.3 (1.43)	8.49	1664.7
Claude-3-5-Sonnet-20240620	79.2 (-1.9, 1.7)	52.4 (1.47)	8.81	1619.9
GPT-4o-2024-05-13	79.3 (-2.1, 2.0)	57.5 (1.47)	8.74	1752.2

Prerequisites for Your System:

Make sure you have the following:

✅GPUs: A100 80GB or H100 (for smooth execution).
✅Jupyter Notebook installed and ready to use.
✅Disk Space: 150 GB free.
✅RAM: At least 128 GB.

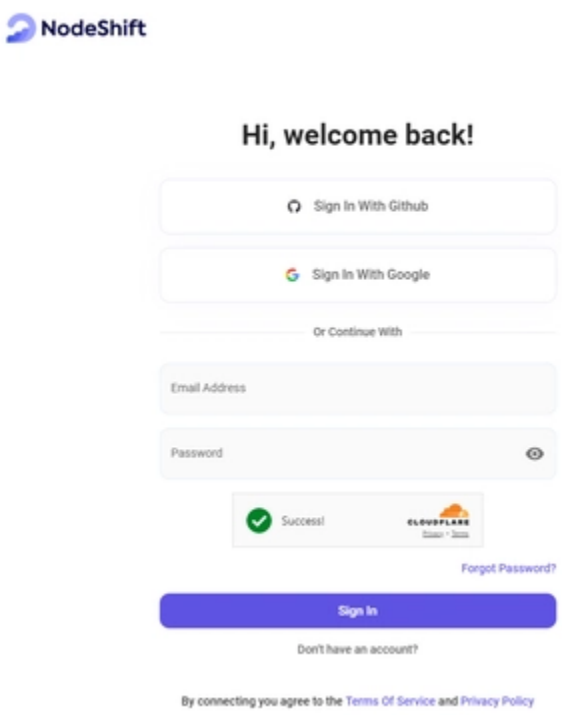
Step-by-Step Process to deploy Llama-3.1-Nemotron-70B-Instruct in the Cloud

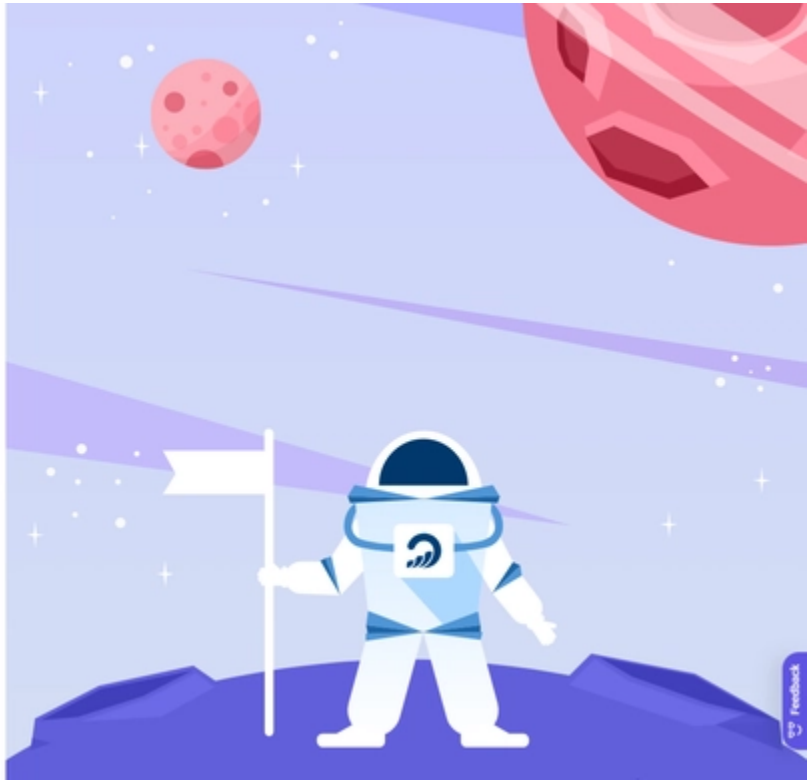
For the purpose of this tutorial, we will use a GPU-powered Virtual Machine offered by NodeShift; however, you can replicate the same steps with any other cloud provider of your choice. NodeShift provides the most affordable Virtual Machines at a scale that meet GDPR, SOC2, and ISO27001 requirements.

Step 1: Sign Up and Set Up a NodeShift Cloud Account

Visit the NodeShift Platform and create an account. Once you've signed up, log into your account.

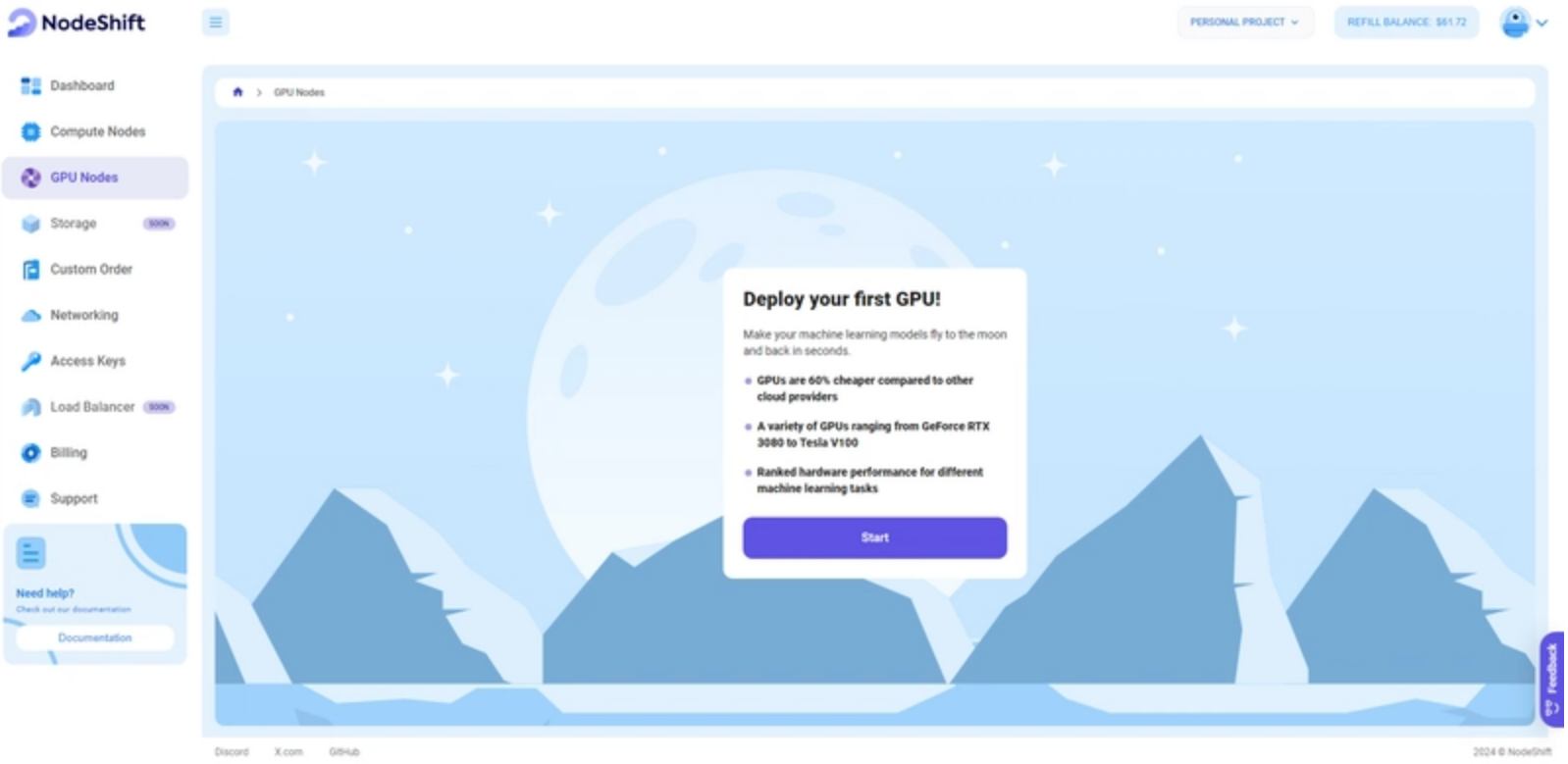
Follow the account setup process and provide the necessary details and information.





Step 2: Create a GPU Node (Virtual Machine)

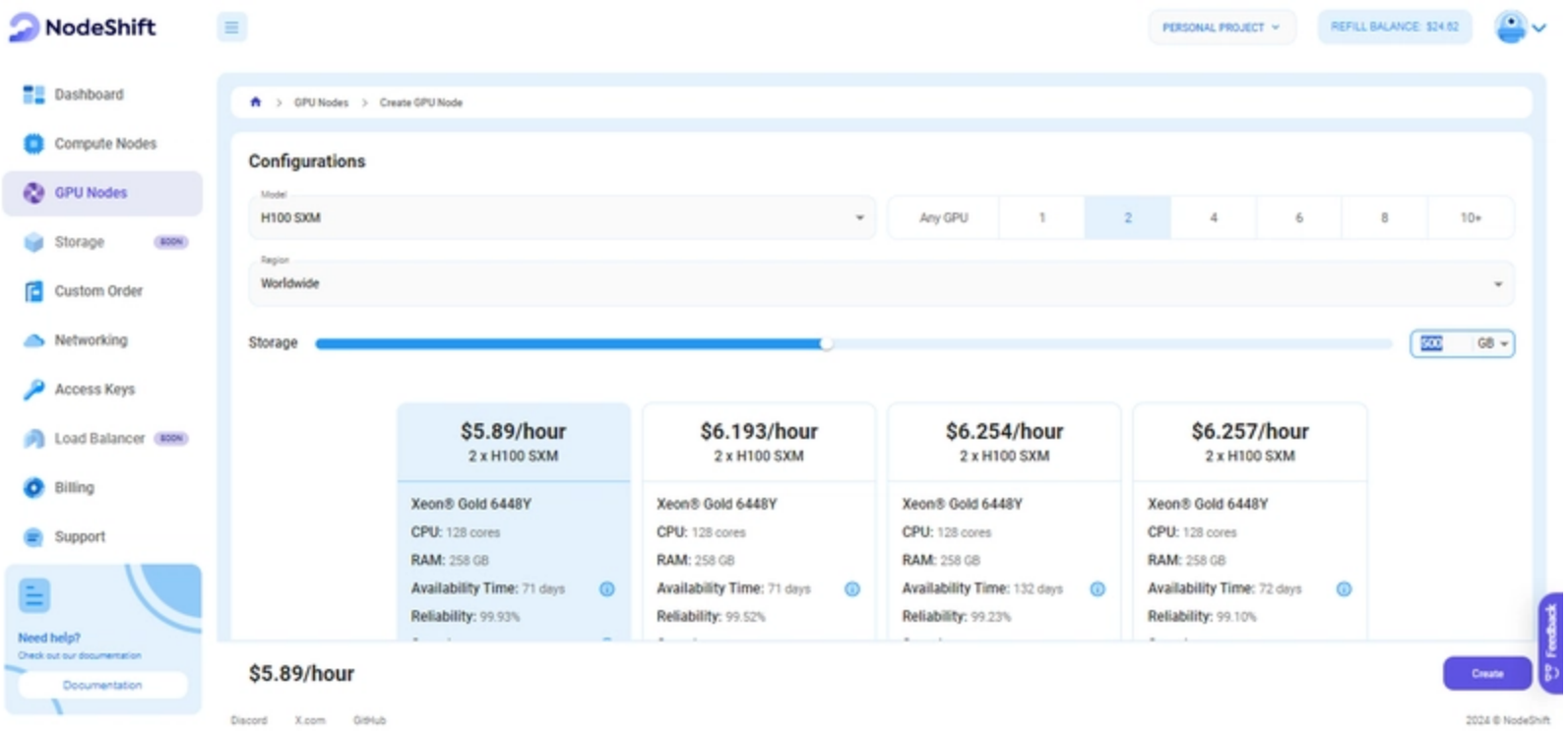
GPU Nodes are NodeShift's GPU Virtual Machines, on-demand resources equipped with diverse GPUs ranging from H100s to A100s. These GPU-powered VMs provide enhanced environmental control, allowing configuration adjustments for GPUs, CPUs, RAM, and Storage based on specific requirements.



Navigate to the menu on the left side. Select the GPU Nodes option, create a GPU Node in the Dashboard, click the Create GPU Node button, and create your first Virtual Machine deployment.

Step 3: Select a Model, Region, and Storage

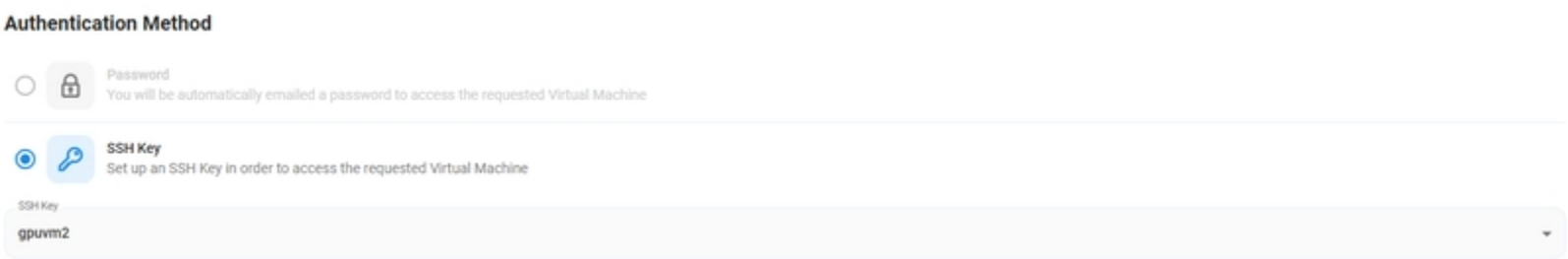
In the "GPU Nodes" tab, select a GPU Model and Storage according to your needs and the geographical region where you want to launch your model.



We will use 2x H100 SXM GPU for this tutorial to achieve the fastest performance. However, you can choose a more affordable GPU with less VRAM if that better suits your requirements.

Step 4: Select Authentication Method

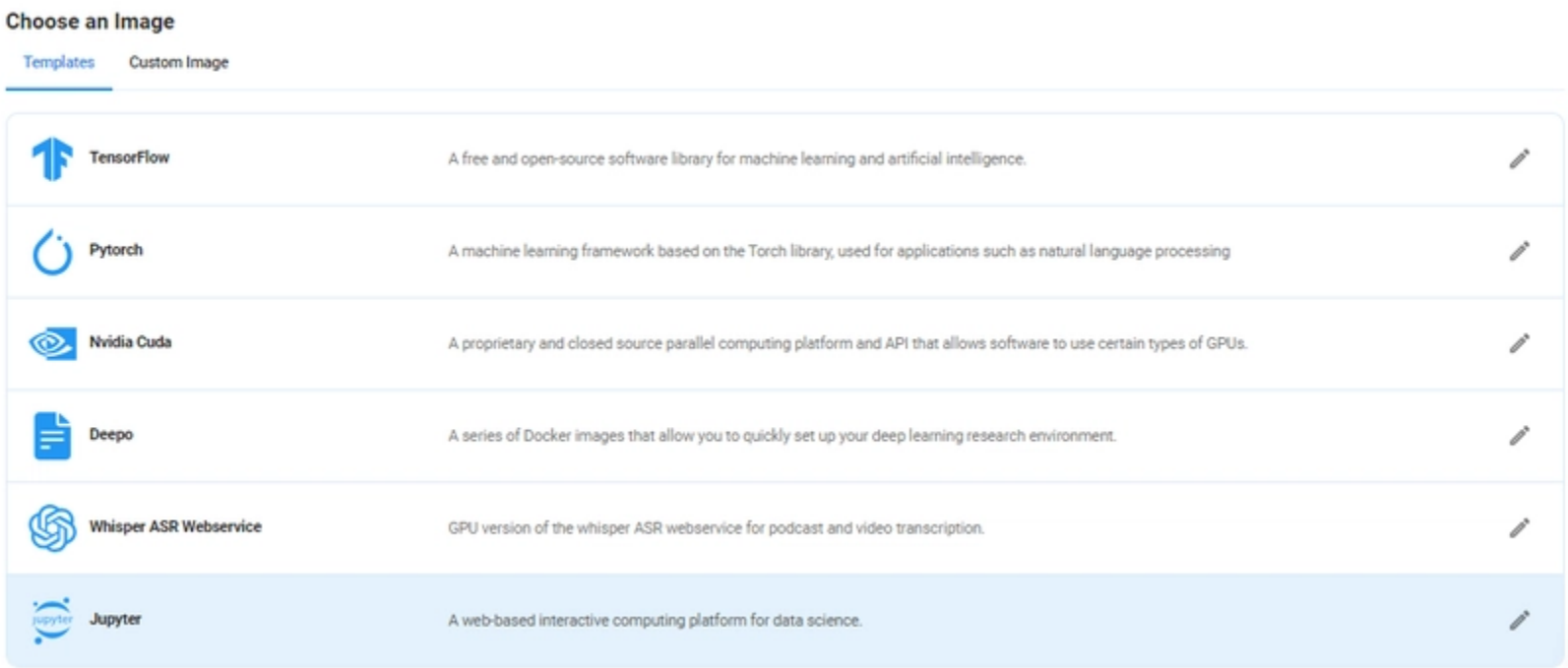
There are two authentication methods available: Password and SSH Key. SSH keys are a more secure option. To create them, please refer to our official documentation.



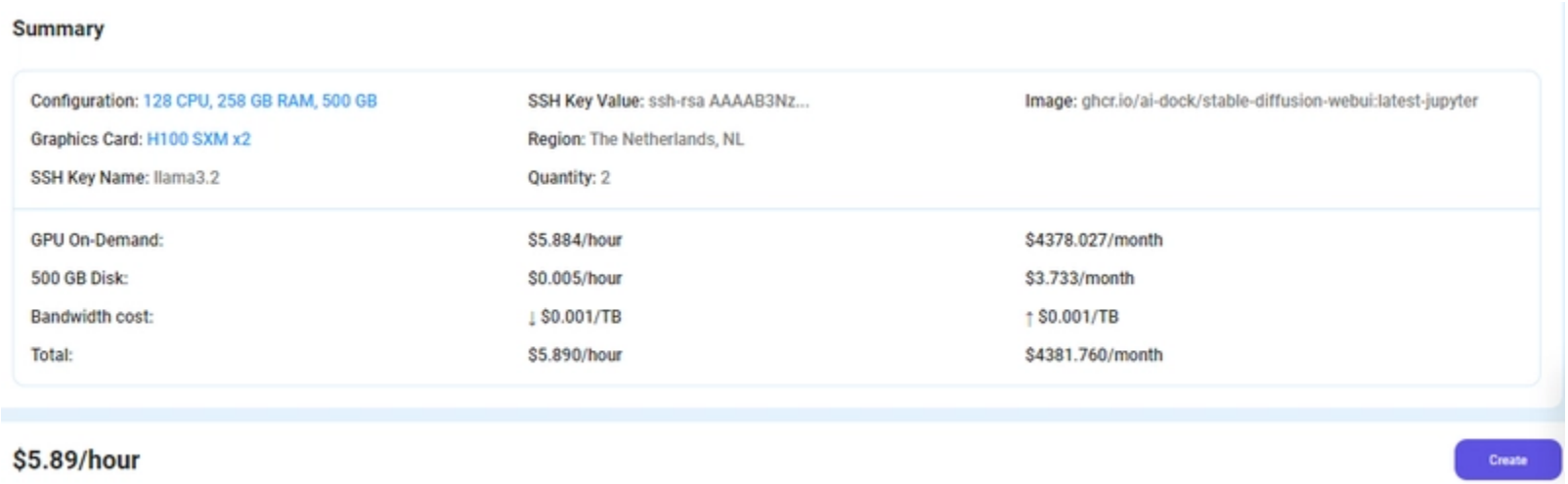
Step 5: Choose an Image

Next, you will need to choose an image for your Virtual Machine. We will deploy the Llama-3.1-Nemotron-70B-Instruct Model on a Jupyter Virtual Machine. This open-source platform will allow you to install and run the Llama-3.1-Nemotron-70B-Instruct Model on your GPU node. By running this model on a Jupyter Notebook, we avoid using the terminal, simplifying the process and reducing the setup time. This allows you to configure the model in just a few steps and minutes.

Note: NodeShift provides multiple image template options, such as TensorFlow, PyTorch, NVIDIA CUDA, Deepo, Whisper ASR Webservice, and Jupyter Notebook. With these options, you don't need to install additional libraries or packages to run Jupyter Notebook. You can start Jupyter Notebook in just a few simple clicks.

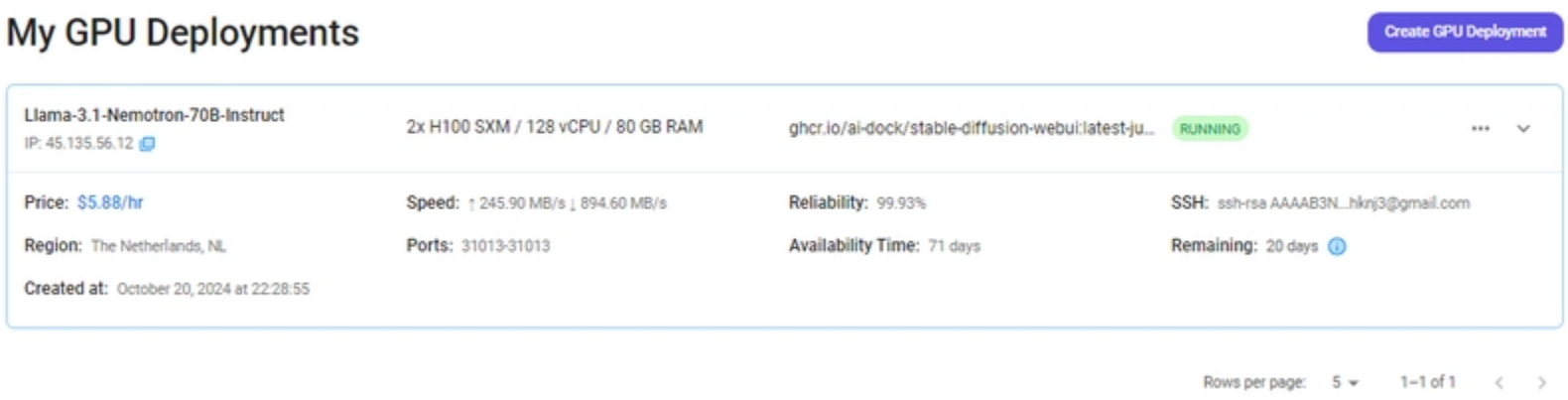


After choosing the image, click the 'Create' button, and your Virtual Machine will be deployed.



Step 6: Virtual Machine Successfully Deployed

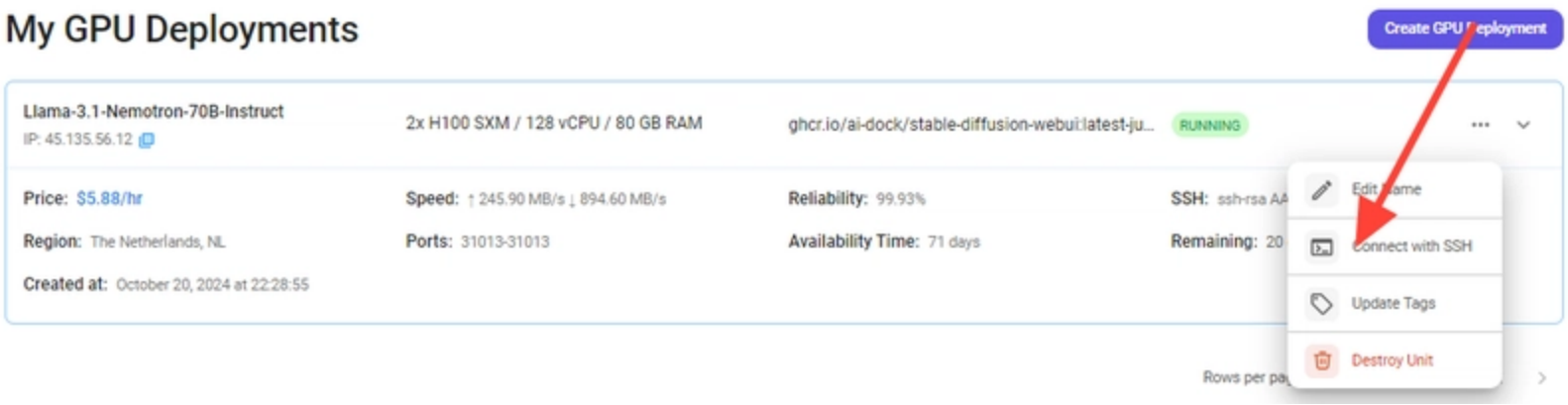
You will get visual confirmation that your node is up and running.



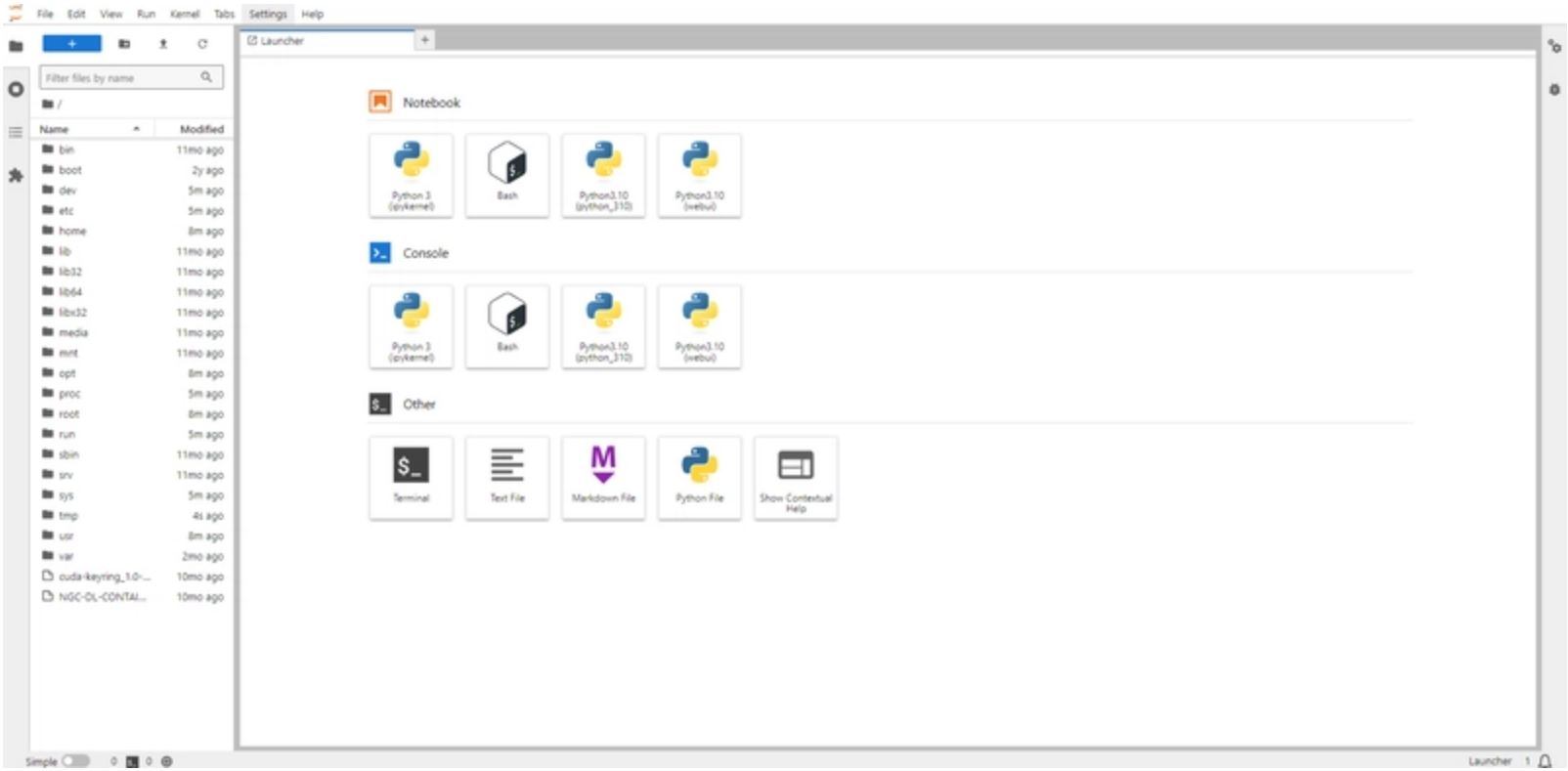
Step 7: Connect to Jupyter Notebook

Once your GPU Virtual Machine deployment is successfully created and has reached the 'RUNNING' status, you can navigate to the page of your GPU Deployment Instance. Then, click the 'Connect'

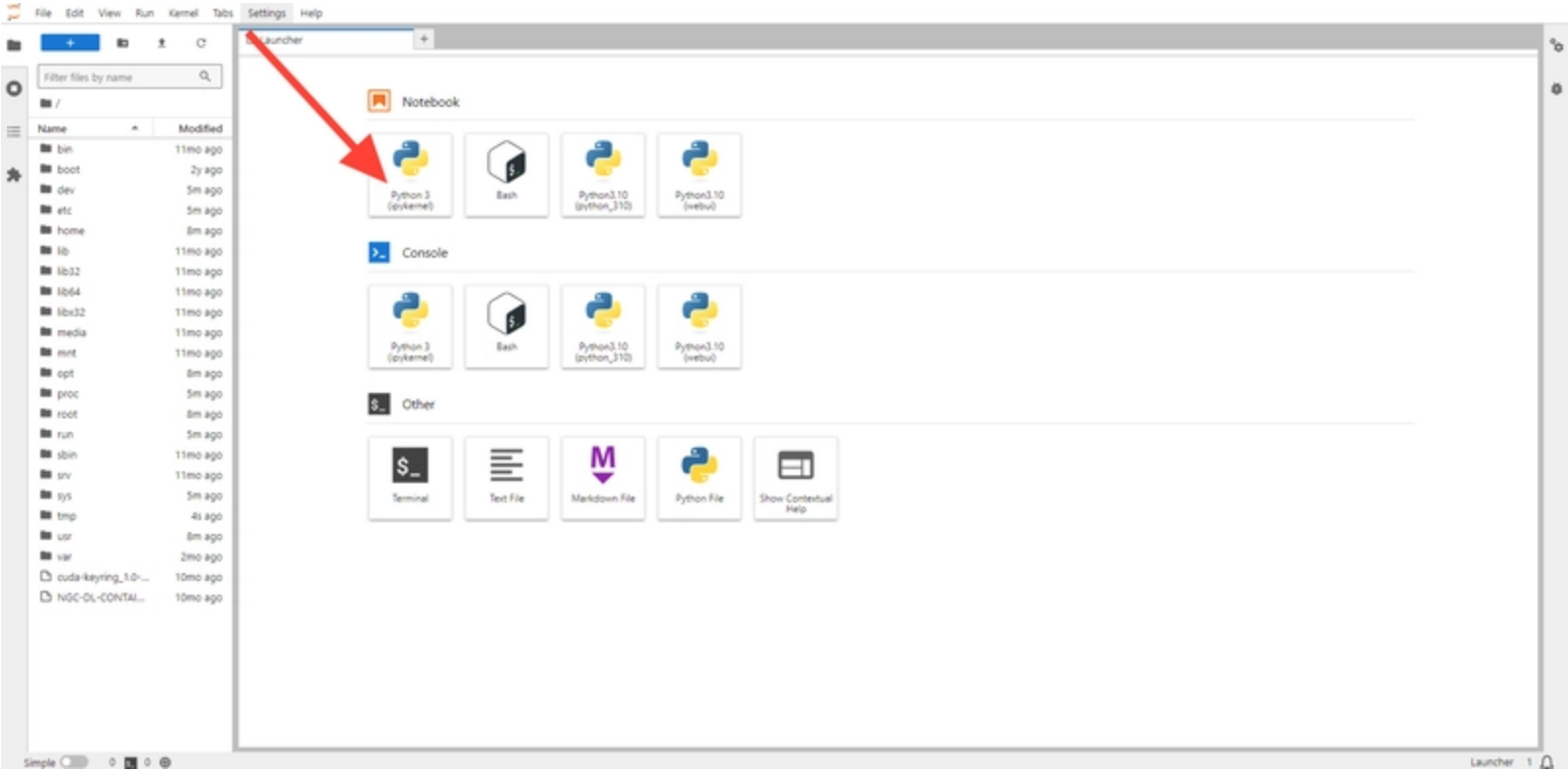
Button in the top right corner.



After clicking the 'Connect' button, you can view the Jupyter Notebook.

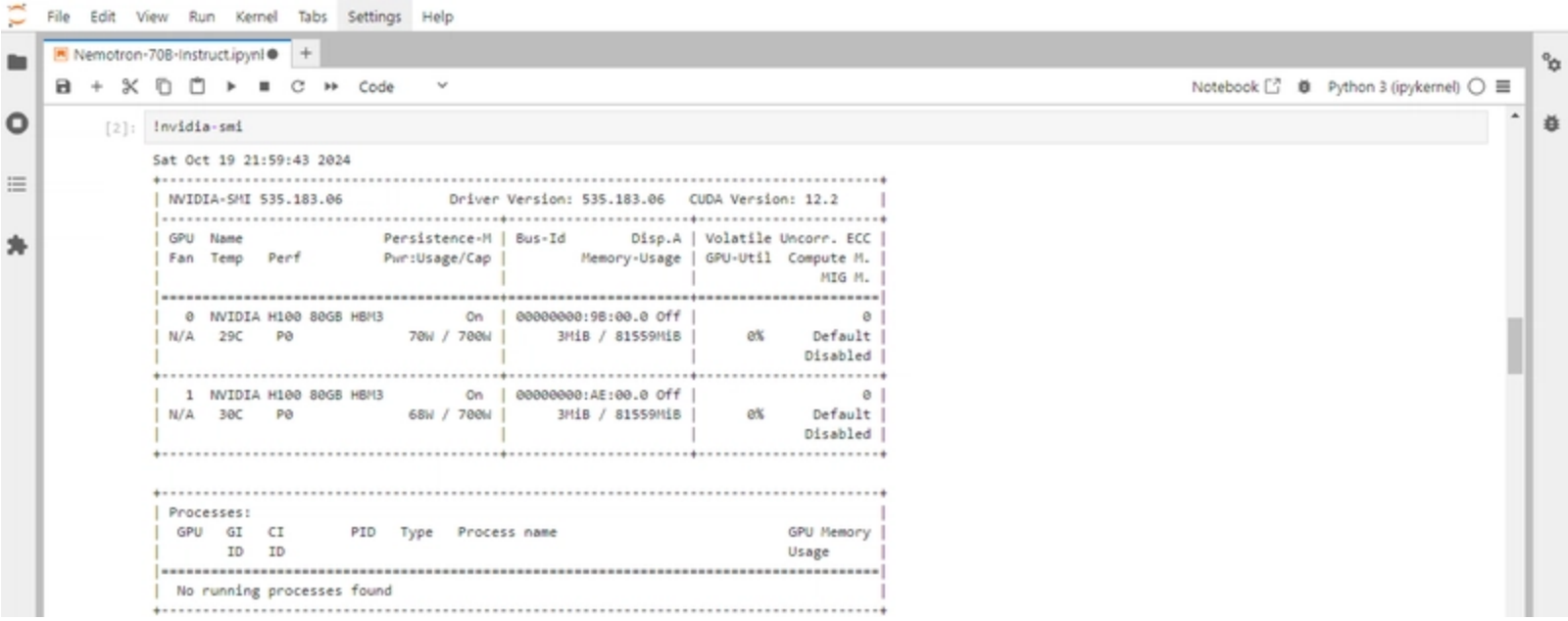


Now open Python 3(pykernel) Notebook.



Next, If you want to check the GPU details, run the command in the Jupyter Notebook cell:

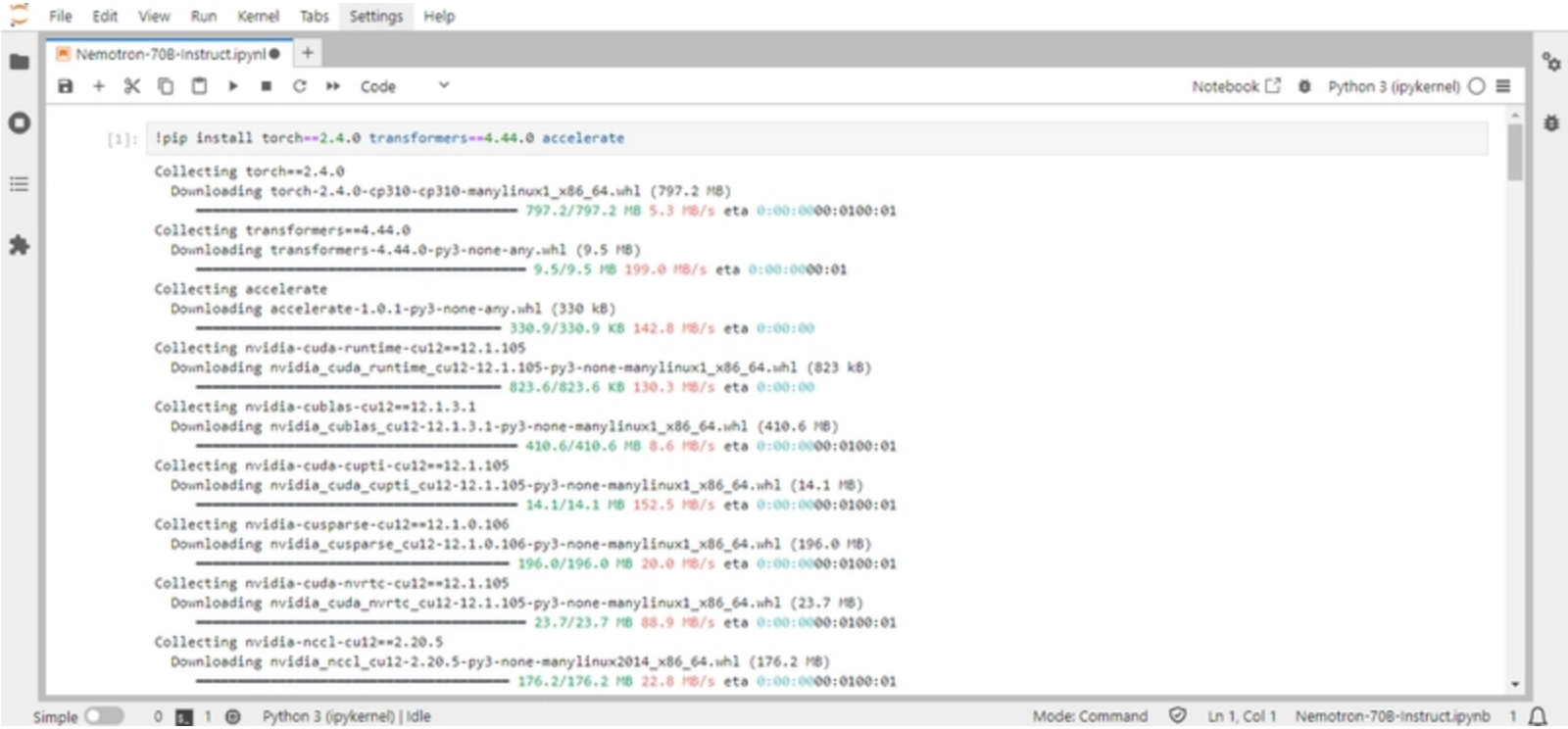
```
!nvidia-smi
```



Step 8: Install the Required Packages and Libraries

Run the following command in the Jupyter Notebook cell to install the required Packages and Libraries:

```
!pip install torch==2.4.0 transformers==4.44.0 accelerate
```



Transformers: Transformers provide APIs and tools to download and efficiently train pre-trained models.

Torch: Torch is an open-source machine learning library, a scientific computing framework, and a scripting language based on Lua. It provides LuaJIT interfaces to deep learning algorithms implemented in C. Torch was designed with performance in mind, leveraging highly optimized libraries like CUDA, BLAS, and LAPACK for numerical computations.

Accelerate: Accelerate is a library that enables the same PyTorch code to be run across any distributed configuration by adding just four lines of code.

Step 9: Load and Run the Model in Jupyter Notebook

Run the following model code in the Jupyter Notebook to load and run the model:

`import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

Model name

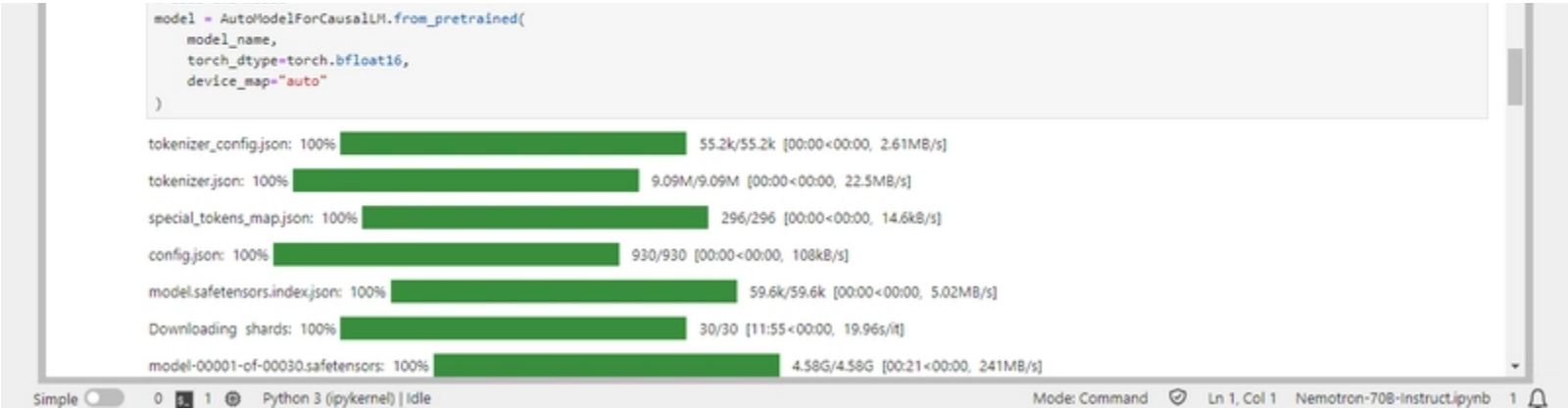
model_name = "nvidia/Llama-3.1-Nemotron-70B-Instruct-HF"

Load tokenizer

tokenizer = AutoTokenizer.from_pretrained(model_name)

Load the model

model = AutoModelForCausalLM.from_pretrained(
model_name,
torch_dtype=torch.bfloat16,
device_map="auto"
)
`



Step 10: Generate Responses in Jupyter Notebook

Run the following code in Jupyter Notebook to generate the output:

`prompt = "How many r in strawberry?"
messages = [{"role": "user", "content": prompt}]

Tokenize input

tokenized_message = tokenizer.apply_chat_template(
messages, tokenize=True, add_generation_prompt=True,
return_tensors="pt", return_dict=True
)

Generate response

response_token_ids = model.generate(
tokenized_message['input_ids'].cuda(),

attention_mask=tokenized_message['attention_mask'].cuda(),
max_new_tokens=4096,
pad_token_id=tokenizer.eos_token_id
)

Decode and display the result

generated_tokens = response_token_ids[:, len(tokenized_message['input_ids'][0]):]
generated_text = tokenizer.batch_decode(generated_tokens, skip_special_tokens=True)[0]

print(generated_text)`

Example 1:

```
1. S
2. T
3. R
4. A
5. W
6. B
7. E
8. R
9. R
10. Y

There are **3 "R"s** in the word "strawberry".
```

Example 2:

File Edit View Run Kernel Tabs Settings Help

Nemotron-70B-Instruct.ipynb

Code | Notebook | Python 3 (ipykernel)

[5]:
```python
prompt = "How many N in NodeShift?"
messages = [{"role": "user", "content": prompt}]

# Tokenize input
tokenized_message = tokenizer.apply_chat_template(
    messages, tokenize=True, add_generation_prompt=True,
    return_tensors="pt", return_dict=True
)

# Generate response
response_token_ids = model.generate(
    tokenized_message['input_ids'].cuda(),
    attention_mask=tokenized_message['attention_mask'].cuda(),
    max_new_tokens=4096,
    pad_token_id=tokenizer.eos_token_id
)

# Decode and display the result
generated_tokens = response_token_ids[:, len(tokenized_message['input_ids'][0]):]
generated_text = tokenizer.batch_decode(generated_tokens, skip_special_tokens=True)[0]

print(generated_text)
```

File Edit View Run Kernel Tabs Settings Help

Nemotron-70B-Instruct.ipynb

Code | Notebook | Python 3 (ipykernel)

```
A simple question!

Let's break down the word "NodeShift":

1. **Node**: contains 1 "N"
2. **Shift**: contains 1 "N" (at the end, but still counts!)

Adding them together: 1 (from "Node") + 1 (from "Shift") = **2**

Answer: There are **2 N's** in "NodeShift".
```

[ ]:

Conclusion

Llama-3.1-Nemotron-70B-Instruct is a groundbreaking open-source model from NVIDIA that brings state-of-the-art AI capabilities to developers and researchers. Following this step-by-step guide, you can quickly deploy Llama-3.1-Nemotron-70B-Instruct on a GPU-powered Virtual Machine with NodeShift, harnessing its full potential. NodeShift provides an accessible, secure, affordable platform to run your AI models efficiently. It is an excellent choice for those experimenting with Llama-3.1-Nemotron-70B-Instruct and other cutting-edge AI tools.

For more information about NodeShift:

Website
Docs
LinkedIn
X
Discord
daily.dev